



浙江大学
ZHEJIANG UNIVERSITY

稳定支付
StablePay

Stablepay Agentic Payment

汇报人: 包博文 贾越 杨世博

指导老师: 谭俊



目录页

Contents

Page

I. 项目背景

II. 技术方案

III. 最终成果

IV. 总结与展望

1.1 项目背景

传统金融支付体系完全围绕人类身份设计，依赖银行开户、人工KYC、绑卡验证等流程，存在账户体系复杂、资金托管风险高、跨境支付费率高、结算滞后、出金困难等诸多痛点。

AI Agent 作为**新型智能执行主体**，无身份证、无银行账户，可自主替人类完成采购、工具调用、付费订阅等操作，行业亟需一套适配 AI 的**去中心化、无人化支付体系**。



➤ 当 AI Agent 从对话工具升级为自主消费主体时，如何重构适配 AI 生态的**支付底层逻辑**？





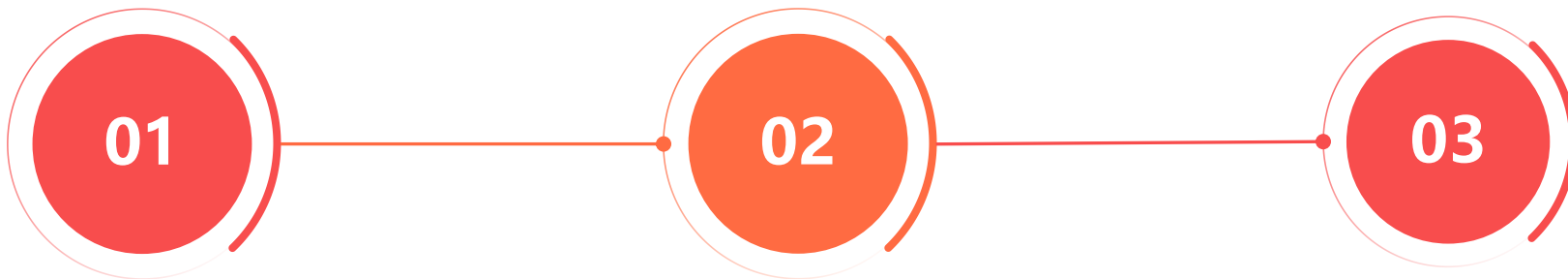
1.2 技术概念

区别于传统人工支付模式，**专为AI Agent打造**的自动化支付体系，无需人工干预、无需人类身份校验，支持AI**自主**发起交易、校验订单、完成结算。

依托**Solana公链USDC稳定币**开展交易，解决传统跨境支付汇率波动、清算周期长、跨境受限问题，实现**秒级链上结算、低成本资金流转**，适配AI Skill轻量化付费场景。

Agentic Payment

稳定币



x402协议

基于**HTTP 402 Payment Required**迭代的轻量化智能支付协议，为本项目核心通信标准。
可实现自动化支付拦截、交易校验、状态回调，适配AI工具按需付费、小额高频的交易场景。



1.3 项目定位与切入点

项目定位

为 Agent 提供一套可自主发起、可验证、可结算的稳定币支付底座，让支付不再依赖人类身份和人工确认流程。

第一切入场景

从 ClawHub 切入，解决 Skill 作者希望获得报酬、Agent 自动购买工具和能力的问题（后续扩展到其他数字资源卖家）

生态扩展方向

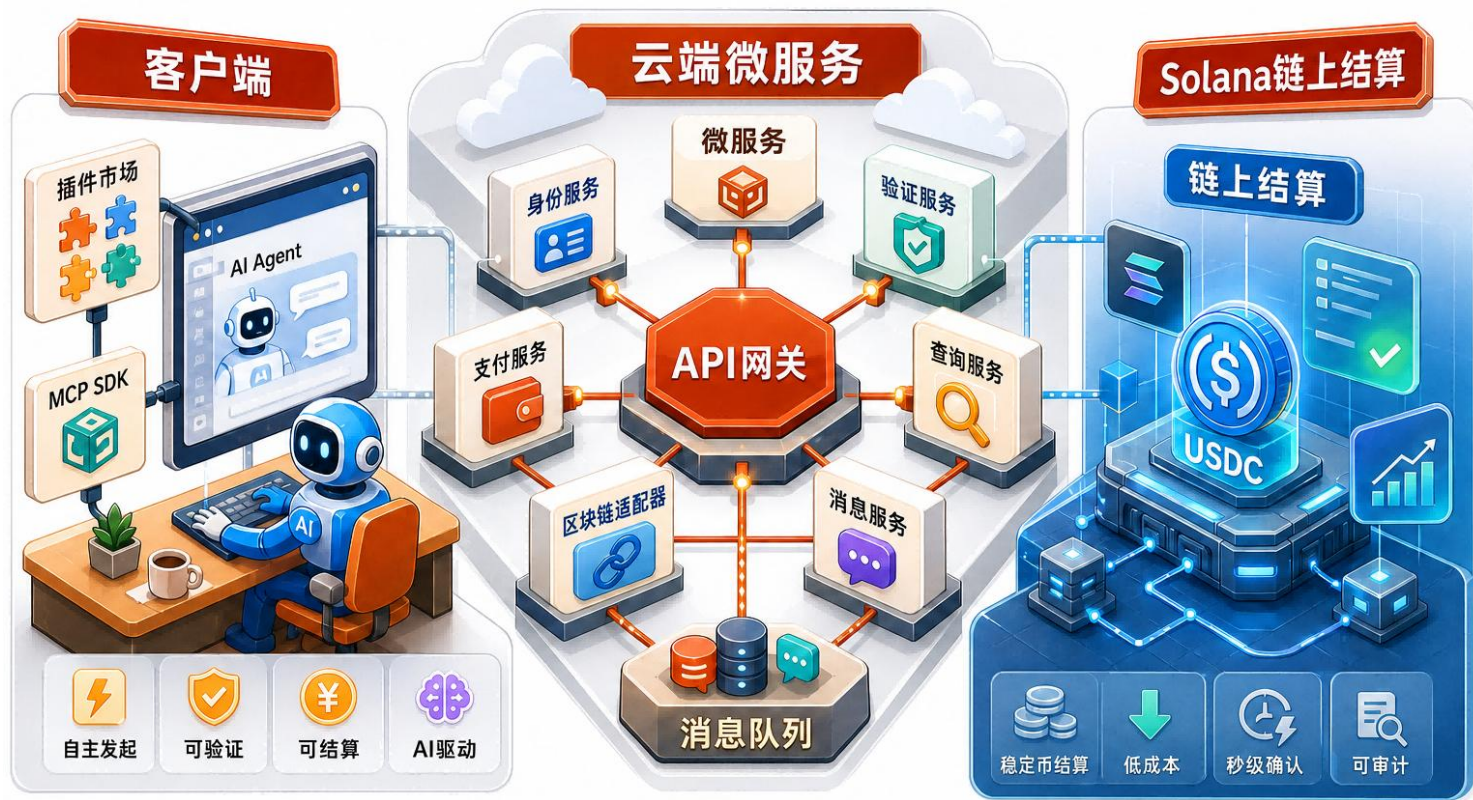
双轨支持 OpenClaw 插件与 MCP SDK，后端从服务单一宿主升级为服务整个 Agent 生态。

我们的目标不是再做一个收银台，而是把 AI 如何支付 抽象成一套底层逻辑：谁可以付、何时可以付、付完如何验证与复用。

一句话总结：StablePay 希望成为 Web3 时代面向 AI Agent 的 PayPal 级支付基础设施。

2. 核心技术方案

- 本项目搭建**客户端AI插件+Cloudwego全家桶微服务后端+阿里云云端部署流水线**的全链路技术架构，从IDL接口规范出发，实现开发标准化、服务解耦化、部署自动化，完整覆盖AI Agent支付全流程能力。



三层设计

客户端层

本地钱包、DID、额度策略

服务层

网关、支付、验证、查询协作

结算层

Solana + USDC 完成最终转账



打造面向AI Agent的
自主、可信、可结算的支付底座

让 Agent 的每一笔支付都「能做、敢做、查得清」

2.1 后端微服务

- 后端不是一个「大接口」，而是把身份、支付、查询、验单拆成独立服务，再用 **API Gateway** 和消息队列把它们组织起来。

同步调用

网关分发后，关键链路通过 Kitex RPC 快速完成。

异步解耦

支付成功后发布事件，验证服务异步落购买记录。

工程价值

单个服务职责更清晰，也更适合后续扩容与迭代。

六大服务协同，驱动 AI Agent 支付全流程



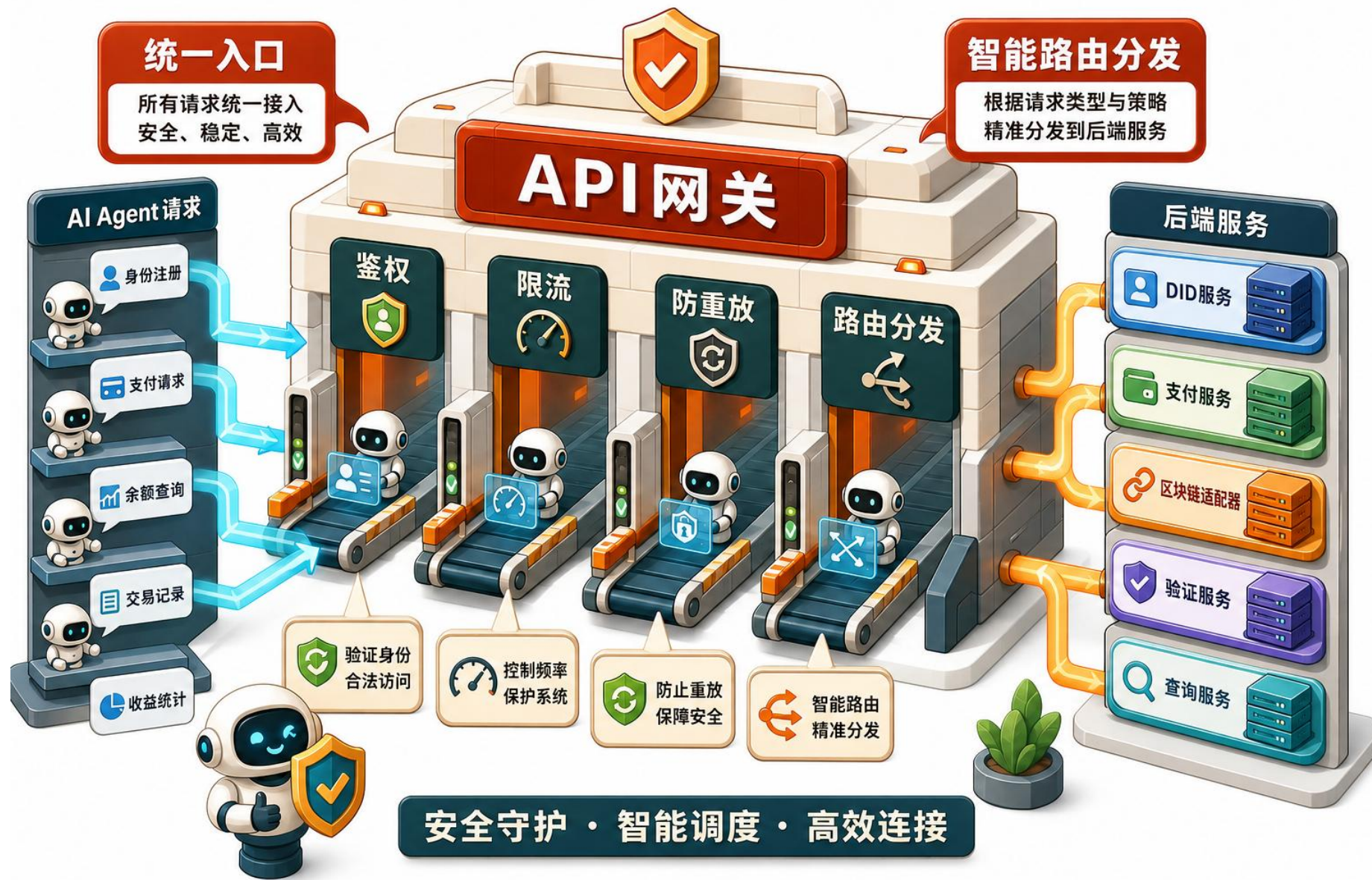
2.1.1 API Gateway

服务职责

API Gateway 是 StablePay 后端的**统一接入入口**，负责把所有外部请求安全、稳定地**转发到具体微服务**。

关键能力

- `/api/v1/{did,payment,verify,query,revenue,sales}` **21 条核心路由**的核心承接；
- 按路径和方法分发请求，让客户端始终**只面对一个入口**；
- 将鉴权、限流、防重放等横切逻辑集中治理，避免下游服务重复实现。



2.1.2 DID Service

- DID Service 为 AI Agent **生成并管理去中心化身份**，DID 不是传统账号，而是**由密钥控制权定义的身份**；
在 StablePay 中，`did:solana:<公钥>` 直接把 Solana 公钥映射为 Agent 的去中心化身份。

生成 Ed25519 密钥对并拼装
`did:solana:<pubkey>` 身份标识

提供身份注册、查询、验签、状态管理等基础能力

所有支付请求都可先由 DID 服务完成签名校验

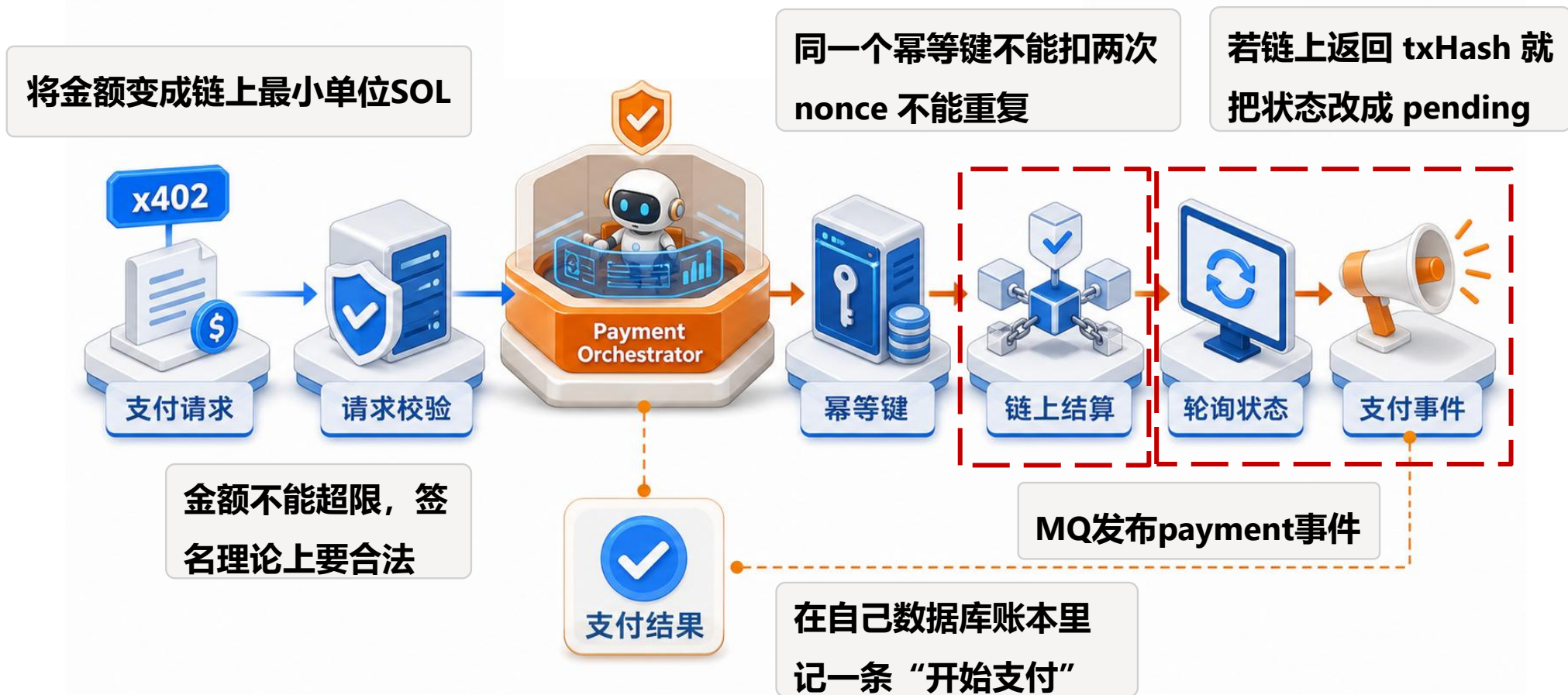


我们的 W3C DID Document 包装了 Solana 公钥，并提供状态与验签服务。

- DID Service 负责 DID 注册、状态管理与通用验签，不关心具体支付业务，只验证某个 DID 是否真的对某条消息做了合法签名。
- 通过**消息 + 时间戳 + nonce**的验签模型，DID Service 为支付系统提供可验证、可防重放、可撤销的身份基础设施。

2.1.3 Payment Service

- Payment Service 是**支付链路核心**，负责处理 x402 协议、协调链上转账，并保证交易可重试、不可重复扣款。一笔支付进来后，它按这个顺序编排：



2.1.4 Blockchain Adapter

➤ Blockchain Adapter 把**业务支付语义**翻译成 Solana 链上动作，是系统与真实稳定币网络交互的执行层。



1. 客户端本地组交易草稿
组出一笔 SPL Transfer:
从买家 ATA到卖家 ATA

authority 是买家

fee payer 热钱包

2. 客户端本地签交易信息
`tx.serializeMessage()`

买家已签名

热钱包未签名

3. 服务端收到部分签名交易

如果 `signed_tx_base64` 存在，说明这是用户已授权的交易，进入 `client_partial_signed` 模式

4. 热钱包只补 fee payer 签名

HotWallet 会先检查交易第一个 account key 必须真的是热钱包自己，然后返回私钥，做 partial sign

2.1.5 Verification Service

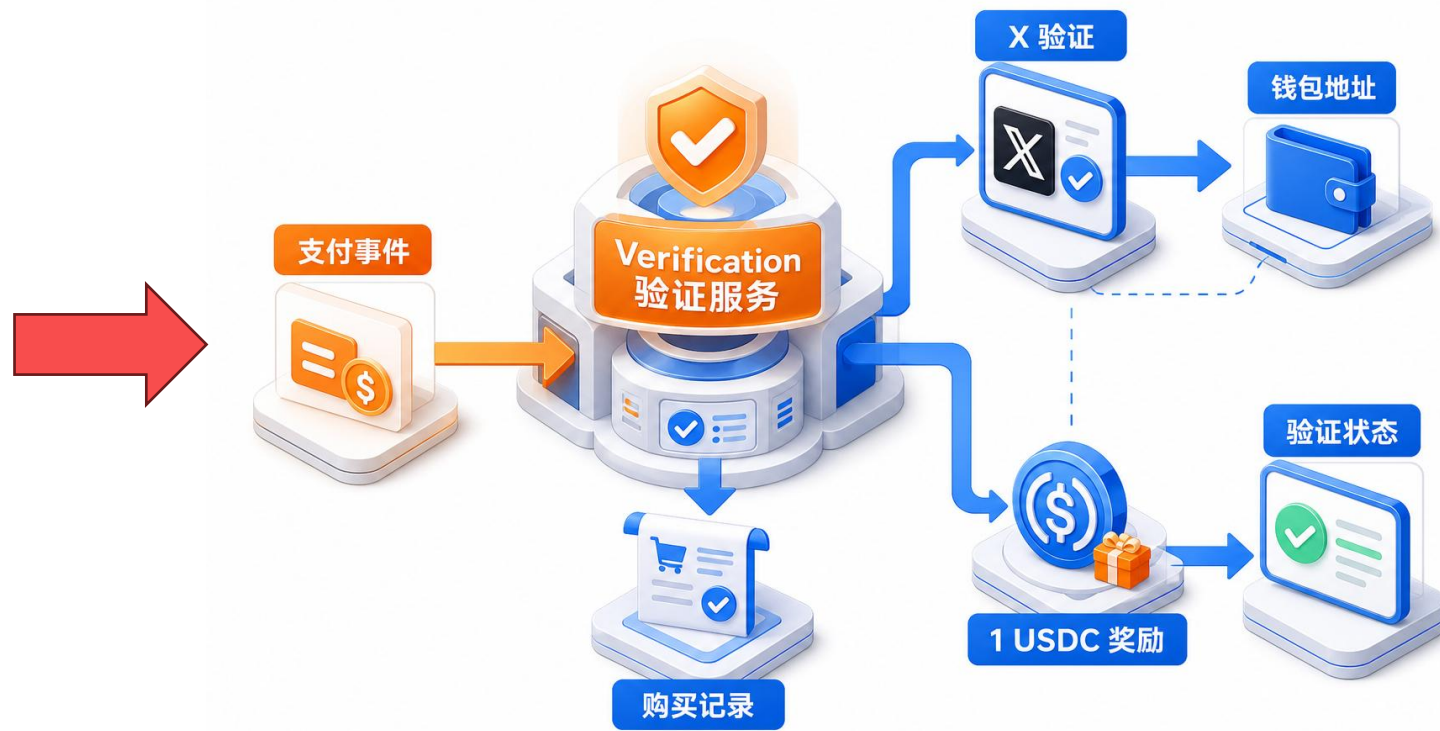
服务职责

Verification Service 负责把链上支付转化为平台已购买的可信证明，同时承担 X 验证与新用户激励发放。

订阅 payment_events, 把 agent 与 skill 的购买关系写入 purchase_records

对外提供单次验证、批量验证、购买证明查询等 RPC 能力

实现 X 推文绑定验证与 1 USDC onboarding 奖励发放



2.1.6 Query Service

```
(base) bubblevan@Bubbles:/mnt/d/MyLab/StablePay/stablepay-openssl-plugin$ kubectl -n zheda-agent exec -it deploy/stablepay-blockchain-adapter -- sh -c 'busybox wget -qO- \
--header="Content-Type: application/json" \
--post-data="{"jsonrpc": "2.0", "id": 1, "method": "getTokenAccountsByOwner", "params": [{"2gLSthKBP2ZWX9chgCHfEqlcG2GT76CimzqFqXXF", ("mint": "4zMMC9srt5Ri5X14GAgXhaHii3GnPAERYPjgZJ0ncDU"), {"encoding": "jsonParsed"}]}]" \
"https://proxy.stablepay.co/https://api.devnet.solana.com"
wget: server returned error: HTTP/1.1 405 Method Not Allowed
command terminated with exit code 1
```

```
(base) bubblevan@Bubbles:/mnt/d/MyLab/StablePay/stablepay-openssl-plugin$ kubectl -n zheda-agent exec -it deploy/stablepay-blockchain-adapter -- sh -c 'busybox wget -S -O- "https://proxy.stablepay.co/https://api.devnet.solana.com" 2>&1 | head -n 20
Connecting to proxy.stablepay.co (8.217.230.156:443)
HTTP/1.1 405 Method Not Allowed
wget: server returned error: HTTP/1.1 405 Method Not Allowed
```

打扰了帅杰哥，但是这个代理，无论是GET还是POST，都是返回的405

ChatGPT说：

blockchain-adapter 底层用的是标准 Solana RPC 客户端，直接拿配置里的 endpoint 去发标准 JSON-RPC 请求，比如 GetBalance、GetTokenAccountBalance、GetLatestBlockhash、SendTransaction。当前代理地址能到，但这个 path 上 GET/POST 都 405。结论：不是“有没有办法访问”，而是“这个代理到底暴露了什么契约”

我们想了解，这个代理给程序调用时，准确的 endpoint 是什么？

它是否支持 标准 HTTPS POST JSON-RPC？

能否直接兼容 Solana 客户端这种 rpc.New(endpoint) 的调用方式？

目前你们 Pod 部署在 dev 环境的 ack 集群，所属的 vpc 是：vpc-bp1op2tvahuhr2sf2x003，命名空间是 zheda-agent，

这个 vpc 是有 NAT 网关的，均可以访问公网，不需要自己配置，出口 IP 是：116.62.178.52

在国内，访问 Solana RPC <https://api.devnet.solana.com> 这种外网地址，会被墙，因此我们在香港搭建了一台代理机器，进行一层转发即可，访问 <https://api.devnet.solana.com> 改为访问：<https://proxy.stablepay.co/https://api.devnet.solana.com>

这台机器设置了白名单，允许 116.62.178.52 这个 IP 过来的请求被转发。

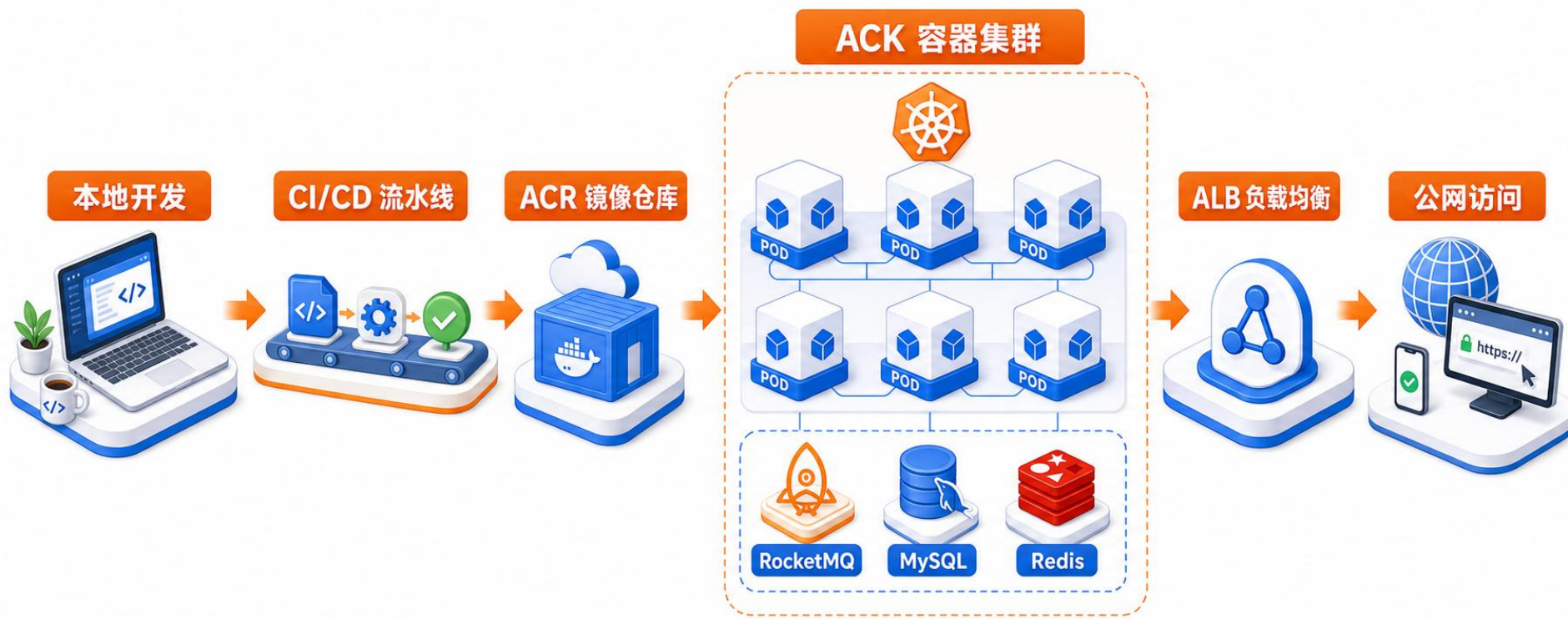
➤ Query Service 面向用户和商家提供余额、交易、收益与销售查询，是支付能力产品化后的可视化出口。

设计亮点：

- 同时服务**买家侧**余额查询和**商家侧**收益、销售明细查询。
- 支持内部交易同步接口，作为**MQ 消费后的幂等写入入口**。

2.2 云端运维

➤ 我们把多微服务系统真正部署到云端，形成从本地开发、镜像构建、集群运行到公网访问的完整交付链路。



ACK 容器编排

统一承载业务服务
与中间件运行

ACR 镜像仓库

镜像统一托管，发
布入口标准化

消息与存储

RocketMQ, MySQL,
Redis 共同支撑链路

公网接入

ALB 统一承接外部
访问与域名流量

2.2 云端运维

- 与此同时我们把发布流程自动化(云效流水线), 并补上了实际运维中的短板。



← stablepay-frontend 最近运行 运行历史 部署历史 统计报表

#8 运行成功

触发信息 包博文 • 页面手动触发 开始时间 2026-06-03 15:29:50 持续时间 1分3秒



stablepay-cluster

容器服务 ACK / 集群列表 / 集群: stablepay-cluster / 无状态 (命名空间: zheda-agent)

无状态 Deployment

名称	命名空间	标签	容器数量	资源用量	镜像	操作
did-service	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/did-servicev1.0	详情 编辑 伸缩
stablepay-api-gateway	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/api-gatewaylatest	详情 编辑 伸缩
stablepay-blockchain-adapter	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/blockchain-adapterv20260604210523	详情 编辑 伸缩
stablepay-did-service	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/did-servicelatest	详情 编辑 伸缩
stablepay-docs	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/stablepay-docsv1.0.7	详情 编辑 伸缩
stablepay-frontend	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/stablepay-frontendv20260618165550	详情 编辑 伸缩
stablepay-merchant-backend	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/merchant-backendv20260613.5	详情 编辑 伸缩
stablepay-mysql	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/mysql8.0	详情 编辑 伸缩
stablepay-payment-service	zheda-agent		1 / 1	CPU MEM	stablepay-registry.cn-shanghai.cr.aliyuncs.com/stablepay-dev/payment-	详情 编辑 伸缩

容器服务 ACK / 实例列表 / 命名空间

stablepay

所在地域 华东2 (上海) 实例规格 经济版 运行状态 运行中

仓库名称	命名空间	仓库状态	仓库类型	风险状态	仓库地址	创建时间	最近更新时间	操作
stablepay-chaingportal	stablepay-dev	正常	私有			2026-07-05 22:45:28	2026-07-06 13:53:33	管理 删除
telepresence	stablepay-dev	正常	私有			2026-06-17 14:11:27	2026-06-17 14:14:39	管理 删除
merchant-backend	stablepay-dev	正常	私有			2026-06-11 22:42:37	2026-06-14 01:25:06	管理 删除
stablepay-cardcore	stablepay-dev	正常	私有			2026-06-08 15:33:50	2026-07-04 14:05:04	管理 删除
curl	stablepay-dev	正常	私有			2026-06-04 16:11:16	2026-06-04 16:12:54	管理 删除
stablepay-docs	stablepay-dev	正常	私有			2026-06-03 10:46:50	2026-06-03 23:40:45	管理 删除
stablepay-chargecore	stablepay-dev	正常	私有			2026-05-22 00:19:50	2026-06-30 17:07:08	管理 删除
stablepay-paymentprod	stablepay-dev	正常	私有			2026-04-15 17:01:25	2026-06-15 10:10:38	管理 删除
nginx	stablepay-dev	正常	私有			2026-04-12 21:40:48	2026-04-12 21:43:45	管理 删除

2.3 Agentic Plugin

- 从 OpenClaw 插件出发，升级为服务整个 Agent 生态的支付客户端，把支付能力产品化成了 Agent 可用、可恢复、可审计、可控风险的一套客户端层。

OpenClaw

已支撑飞书/TUI 场景，证明 Agent 调支付这条路可以跑通。

MCP SDK

目标是让 CC/Codex/Cursor 等宿主都能共享同一套支付后端。

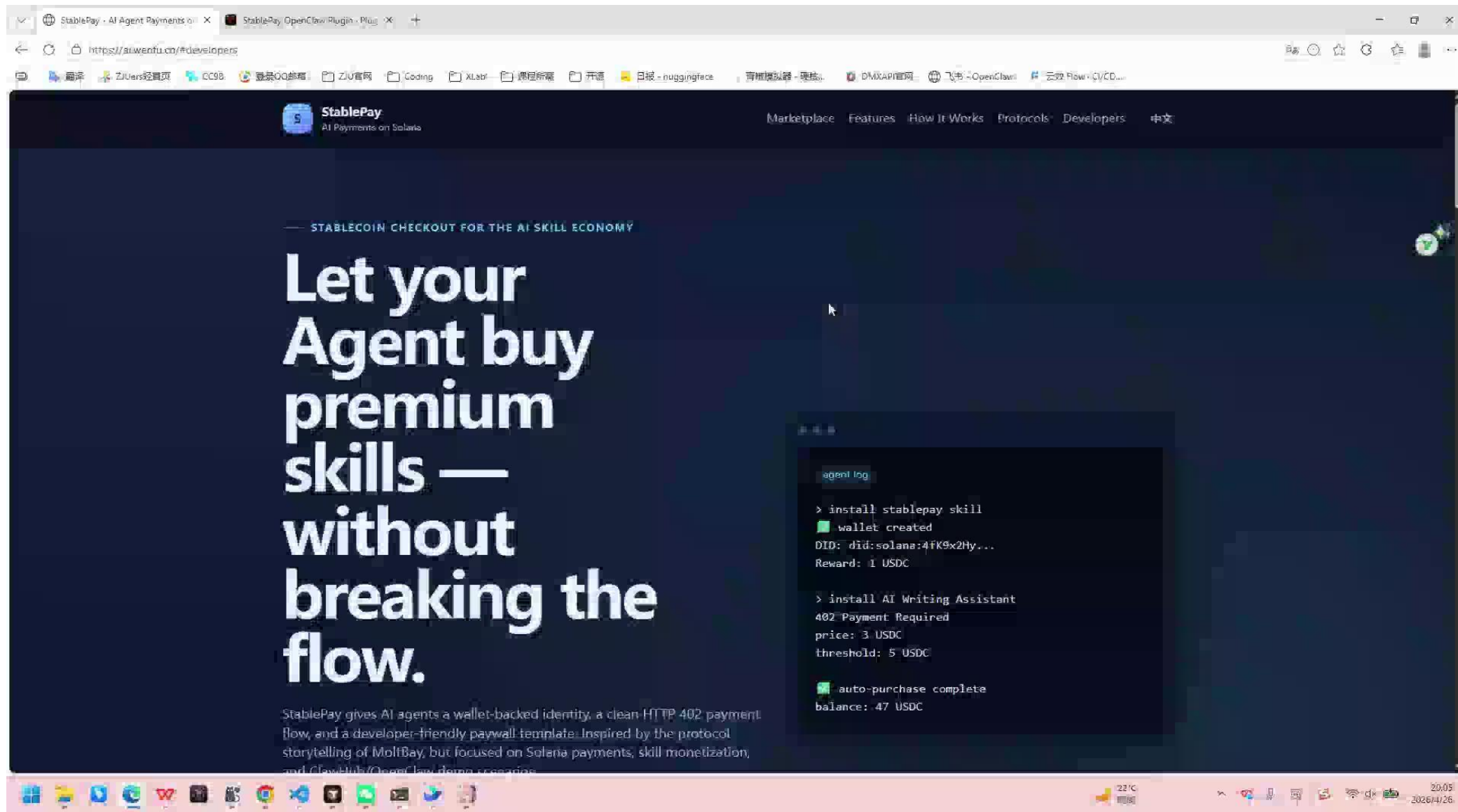
NPX CLI

没有 IDE 宿主时，也能直接调试钱包、DID、限额与支付链路。





2.3.1 客户端演示视频





2.3.2 工具能力矩阵

➤ 当前插件已注册 18 个工具，按**用户任务**分成 4 组：

A. 身份与钱包

stablepay_runtime_status
stablepay_create_local_wallet
stablepay_bind_existing_wallet
stablepay_register_local_did

解决 Agent 拥有“可签名、可持有资产”的本地身份

B. 支付与策略

stablepay_configure_payment_limits
stablepay_build_payment_policy
stablepay_pay_via_gateway
stablepay_query_balance

解决 Agent 在限额内自主支付，并知道自己还能付多少

C. 商家与验证

stablepay_generate_verify_link
stablepay_get_verify_status
stablepay_query_sales
stablepay_merchant_buy_product

解决用户实名门槛、商家接入、已购验证与销售查询

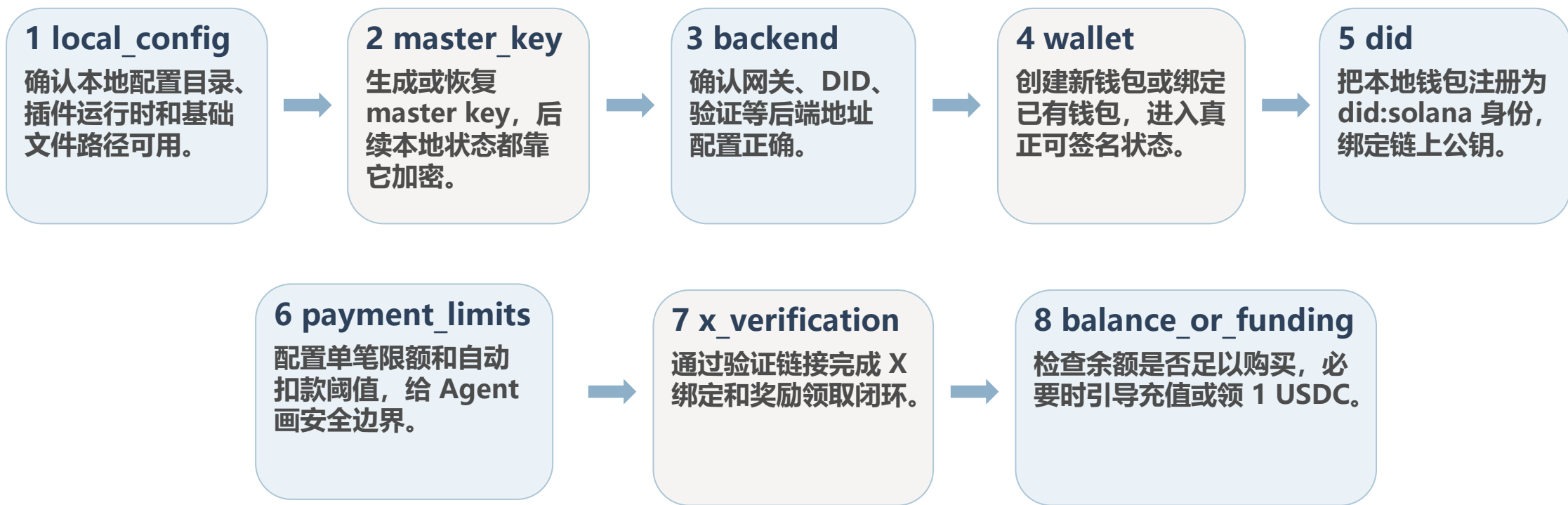
D. 引导与运维

stablepay_doctor
stablepay_onboard
stablepay_operating_manual
stablepay_merchant_get_product

解决首次初始化、状态恢复、故障诊断和日常操作说明

2.3.3 Onboardding 状态机

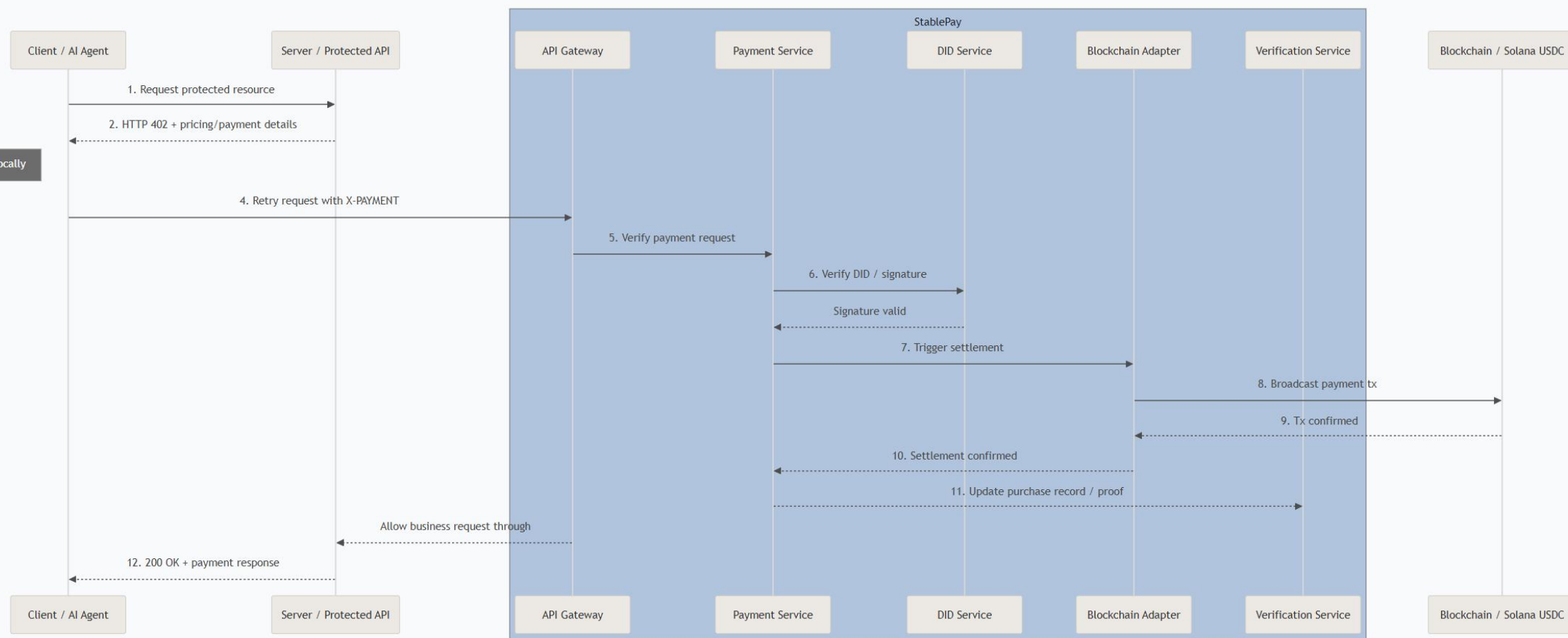
- 在最开始我们遇到了新手指引不明确的问题，在参考openclaw onboard的设计之后重新设计了状态机初始化：不让用户一次填完所有配置，而是更新状态逐步推进；每一步先 doctor，再前进。



stablepay_onboard 返回 onboard_session_id, 24 小时内可继续上次进度，不怕 LLM 上下文被压缩。

2.3.4 Agent 支付全链路

➤ Stablepay 后端作为 X402 生态中的 **Facilitator**，助力 Agentic Payment 生态发展。



2.3.5 插件安全设计

- 让 LLM 自主调用支付工具，最大的风险是 LLM 误判——比如 Claude Code 看到一个工具返回了某些字段，就当“成功”继续往下走，结果其实支付被策略层拒了。为了杜绝误判，OpenClaw 插件层做了三件事：

失败显式返回

- settled.ok=false 时直接返回 failed / policy_denied / manual_confirmation_required
- 避免模型把“有返回”误解成“支付成功”
- 把工具层的失败语义暴露给宿主

大额二次确认

- 超过**单次支付限额**或**自动支付阈值**必须确认
- 没有 confirm_over_threshold 参数就拒签
- 签名前后各做一次策略门校验

本地状态加密

- master key 默认存在 ~/.stablepay-openclaw/master.key
- state 文件用 AES-256-GCM 加密
- debug 日志走 stderr，不污染工具 stdout

质量保障

- eval trace 记录工具选择准确率、参数准确率、任务完成率
- jargon leak 检测避免把底层术语暴露给用户
- 支付工具不是“能用就行”，还要评估体验与安全



3. 最终成果

➤ 我们跑通了“用户用得上、商家接得进、后端撑得住”的三条结果链路：

用户闭环

开户、验证、充值、购买、到账都能走通。

开发者闭环

商家后端可返回 402，并依赖 verify 结果放行。

平台闭环

微服务、消息流与云端部署可稳定演示。

我们一共做了 10 人测试，最有价值的在于看清楚不同用户在什么地方卡住。

用户画像

- 纯小白：文档与 onboarding 是生死线
- 理工科 / Coding 用户：配置能过，但 USDC 获取困难
- 已有 OpenClaw 用户：整体最顺

主要痛点

- onboarding 中会跳过 X 验证
- 奖励到账感知弱，闭环不强
- 大陆用户稳定币使用门槛仍偏高
- 文档对非技术用户不够友好

对应改进

- 新增 x_verification 与 reward_claim 步骤
- 余额轮询确认奖励到账
- 手动水龙头 + 礼品码页面增强激励
- 用 Mintlify + llms.txt + MCP 重写文档入口

3.1 用户侧闭环

➤ 从用户视角看，StablePay 最关键的价值是把“Agent 自己买东西”这件事，做成了可感知的一条顺滑流程。

1 安装插件

用户说一句“帮我初始化 StablePay 插件”。

2 自动开户

创建本地钱包并注册 DID。

3 跳转验证

打开 verify 链接完成 X 绑定。

4 领取奖励

Verify & Claim 后收到 1 USDC。

5 配置限额

设置 50 USDC 总限额与 5 USDC 自动阈值。

6 自动小额购买

购买 3 USDC Skill, Agent 自动完成支付。

7 大额人工确认

购买 15 USDC Skill 时弹确认，再完成扣款。

自动支付场景

- 3 USDC 商品低于自动阈值，客户端直接签名支付
- 支付后余额从 50 降到 47，形成清晰结果反馈

确认支付场景

- 15 USDC 商品超过阈值，不允许 LLM 自己做主
- 用户确认后再执行，余额从 47 降到 32

稳定支付
StablePay

I'm AI User Wallet Guide I'm AI Developer 中文

— GET STARTED

Quick Start Guide

1. Install Plugin & Configure

Install the StablePay plugin in OpenClaw by entering the command below:

```
openclaw plugins install clawhub:stablepay-agentpay-dev@0.3.19 --force --dangerously-force-unsafe-install
```

2. Chat with OpenClaw to Initialize

After installing the plugin, simply chat with OpenClaw to start initialization. OpenClaw will automatically guide you through wallet creation and configuration:

Conversation Example

Here is a real initialization conversation example:

OPENCLAW:

No wallet detected. Would you like to create a new wallet?

Create command:
stablepay_create_local_wallet --user_id your_name

Confirm creation?

OPENCLAW:

Wallet created successfully!
New wallet info:
Address: 6Hhpd8NWDN5D3rt8cGYoR24Fwfcrb2QC4s6Fz6qTq1B

USER:

Help me initialize StablePay plugin

USER:

Create one



3.2 开发者侧闭环

- 我们同时验证了商家接入路径，同时搭建了样例商家后端，把 StablePay 当作一个外部支付底座来调用。

接入步骤

- 开发者创建钱包并注册 Skill DID
- 把 SKILL_DID 写进 merchant 配置
- merchant backend 的 /execute 先调 Verification /verify
- 未购买时返回 402，让客户端走支付重试链路
- 支付完成后重试业务请求返回 200

为什么这条闭环重要

- 商家不需要自己做链上支付，只需要会返回 402 challenge
- StablePay 客户端拦截、签名、支付、重试全部自动完成
- 商家只关心“这个 Agent 有没有买过”，而不是钱包细节
- 这证明 StablePay 可作为支付底层复用

Mintlify 文档站把接入知识产品化

StablePay English

Search...

Ctrl K

Guides API reference

MoltBay OpenClaw GitHub

Getting started

Introduction Quickstart End-to-end testing Plugin configuration

Identity & wallet

Wallet setup DID registration

Payment integration

Integrating x402 payments Backend verification x402 response format

Marketplace

Publishing on MoltBay Revenue & sales Settlement & reconciliation

Getting started

Quickstart

Add StablePay payment to your skill in under 5 minutes

Copy page

Add payment to your skill

Open your skill's `skill.md`. Paste this at the top or bottom — replace two values.

```
## 🧑‍💻 StablePay Payment

This Skill requires {PRICE} USDC to unlock.

Payment endpoint:
https://api.stablepay.co/pay?skill={SKILL_DID}&price={PRICE}

Verify purchase:
https://api.stablepay.co/verify?skill={SKILL_DID}&agent={AGENT_DID}
```

Variable	Replace with
{PRICE}	Your price in USDC (e.g., 5)
{SKILL_DID}	Your developer DID (did:solana:AbCdEf...)
{AGENT_DID}	Leave as-is — the buyer's agent fills this at runtime

💡 Don't have a DID yet? The section below walks you through one-time wallet setup. You only do this once — after that, adding payment to each skill is just the template above.

How it works

```
Buyer's Agent reads your skill.md
→ Sees the payment block
→ Runs stablepay_pay_via_gateway in conversation
→ Signs the SPL transfer with their wallet
→ USDC sent directly to your wallet (skill_did)
→ Buyer's Agent gets access
```

3.3 后端与云上运行

- 集群长期稳定运行，而且公网入口、发布与运维观测已经打通。

集群稳定运行

6 个业务服务和核心中间件统一跑在 ACK 上，关键 Pod 长期保持 Running。

```
(base) bubblevan@Bubbles:~$ kubectl get pods -n zheda-agent
```

NAME	READY	STATUS	RESTARTS	AGE
did-service-654c7f59cd-hns44	1/1	Running	0	93d
stablepay-api-gateway-556c9847b7-z5j4c	1/1	Running	0	86d
stablepay-blockchain-adapter-65794c868b-blwrm	1/1	Running	0	34d
stablepay-did-service-fb5fcfc59-tfc8d	1/1	Running	0	34d
stablepay-docs-7dbc6ff95-4ntb8	1/1	Running	0	34d
stablepay-frontend-5868595cf5-sn7qm	1/1	Running	0	23d
stablepay-merchant-backend-74b555dc67-v5svn	1/1	Running	0	27d
stablepay-mysql-665d697689-jltw5	1/1	Running	0	91d
stablepay-payment-service-7b689ff74b-m7xbw	1/1	Running	0	29d
stablepay-query-service-658b886d4-v9hxs	1/1	Running	0	34d
stablepay-redis-6bc44496bb-zlxgq	1/1	Running	0	91d
stablepay-rocketmq-broker-cb99dcb5b-tfw4v	1/1	Running	0	34d
stablepay-rocketmq-nameserver-8fbb784f-d44pg	1/1	Running	0	91d
stablepay-verification-service-8444b7f98-7fqng	1/1	Running	0	26d

入口与域名打通

ALB Ingress 将 ai.wenfu.cn 暴露到公网，后续滚动发布、扩容和回滚都有统一入口承接流量。

```
(base) bubblevan@Bubbles:~$ kubectl get ingress -n zheda-agent
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
stablepay-ingress	alb	ai.wenfu.cn	alb-3m55g2idsjhrdy3p6y.cn-hangzhou.alb.aliyuncs.com	80	91d



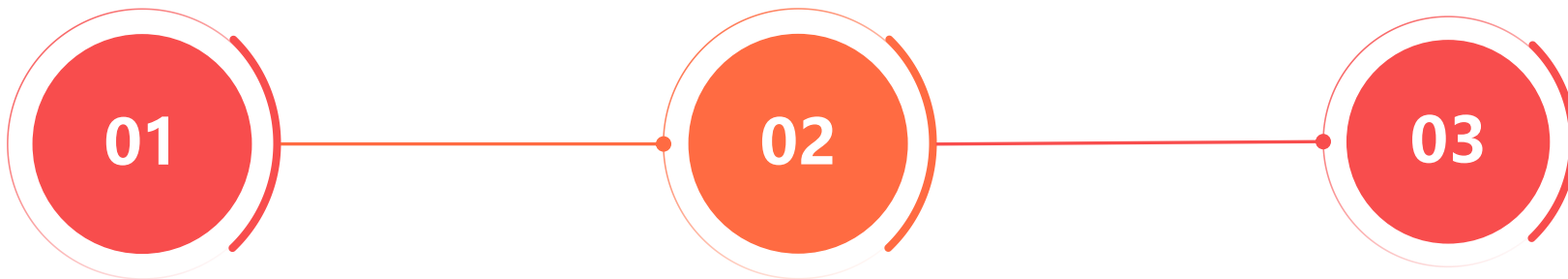
4. 总结与展望

- CloudWeGo: Hertz、Kitex、Thrift 让接口规范先行
- RocketMQ、MySQL、Redis、MongoDB 把支付链路撑起来
- Solana / USDC、ACK / ACR / ALB 让链上与云端真正联动

工程栈第一次串起来

- 从 PRD 出发，先写 IDL，再拆服务、接链、接宿主
- 从 Minikube / 本地调试一路走到 ACK 集群上线
- 从能跑进一步走到能演示、能解释、能迭代

最像工程师的一次体验



学到的是系统协作

- 不是单点写一个接口，而是让网关、支付、验签、验单、查询彼此配合
- 不是本地跑通就算完成，而是要让插件、文档站和云端部署一起闭环
- 不是写完代码就结束，还要会排障、回滚、做可观测与运维



4. 总结与展望

➤ 这套系统能从想法走到上线，不只是因为写了代码，更因为一路上有人一起把它推到了能落地的程度。

感谢提供方向与资源的人

- 感谢 StablePay 公司提供课题方向、PRD、技术方案支持与云资源
- 感谢 Jun Tan 在架构设计、排障思路和资源协调上的深度指导
- 感谢 X-Lab 提供把课堂知识搬进真实生产环境的机会

感谢一起把项目做成的人

- 感谢团队同学从 1 月开工到期末周仍然持续投入
- 感谢大家从一行 Go import、一次 RocketMQ 故障、一次 Ingress 排查开始，把 5 万行工程真正跑起来
- 这一百多天我们真实地做了一回工程师

验证了方向

402 + DID + 稳定币结算，确实能支撑 Agent 购买商品。

暴露了瓶颈

新手 onboarding、资金获取和客户端生态覆盖仍是门槛。

指向了下一步

MCP SDK、capability 复用、更多商家场景会把它推向平台化。

再次感谢 StablePay 给了我们一次 Change the world by programming 的真实实践机会。

请大家批评指正！

汇报人: 包博文 贾越 杨世博

指导老师: 谭俊